FOUNDATION_CAMUNDA - Workflow Engine Foundation

Developer implementation guide: `FOUNDATION_CAMUNDA_DEV_IMPL_GUIDE.md`.

1. Purpose

This foundation defines how SOPHIX uses Camunda for long-running workflows, BPMN orchestration, external task workers, message correlation, timers, and human tasks.

Use this document when implementing:

- A BPMN workflow.
- A Camunda external task worker.
- A user task and candidate-group assignment.
- A message catch event.
- A timer or boundary timer.
- Workflow incident/retry handling.
- Workflow deployment and versioning.

2. Ownership

FOUNDATION_CAMUNDA owns:

- Camunda deployment topology.
- BPMN execution conventions.
- External task worker pattern.
- User task notification pattern.
- Message correlation rules.
- Timer and retry conventions.
- Workflow deployment/versioning conventions.
- Operational recovery expectations.

FOUNDATION_CAMUNDA does not own:

- Business rules for a specific workflow. Those live in the owning DD.
- Worker business logic. Each module owns its workers.
- User/role definitions. See `FOUNDATION_AUTH.md`.
- Event publishing contracts. See `FOUNDATION_KAFKA.md`.
- Module database schemas outside the Camunda database. See each DD.

3. When To Use Camunda

Use Camunda when the workflow has at least one of:

- Human task approval or assignment.
- Multi-step orchestration across modules.
- Waiting on an external event or field operation.
- Timers measured in minutes, hours, or days.
- Retry and incident visibility requirements.
- Business-readable BPMN process shape.

Current SOPHIX examples:

Area	Camunda usage
FUL-02 Order Capture	Sales order capture, validation, payment wait, KYC, install wait, activation handoff
Subscription workflows	Activate, pause, resume, terminate, upgrade, downgrade, relocation, migration, restrict, suspend-NP
WO-01 Work Orders	Installation, shifting, support field flows
OSR-RMA	Equipment swap, pickup, upgrade, RMA flows
Future operational flows	Maker-checker, ticketing, internal reviews

Do not use Camunda for simple CRUD or single-transaction service methods.

4. Architecture

```
flowchart LR
    MOD[Module APIs] -->|start/message/task REST| LB[NGINX or LB]
    WORKERS[Module workers] -->|fetchAndLock / complete| LB
    UI[Backoffice UI via module API] -->|task actions| MOD

    LB --> C1[Camunda node 1]
    LB --> C2[Camunda node 2]
    C1 --> DB[(Camunda PostgreSQL DB<br/>Cluster C)]
    C2 --> DB
```

Camunda is an orchestration engine. It stores workflow state in PostgreSQL and hands business work to module workers through external tasks. Modules do the actual business work.

5. Deployment Topology

Item	Requirement
Camunda version	Camunda 7 runtime unless a later project decision changes the engine
Nodes	2 Camunda nodes
Front door	NGINX or load balancer
State	PostgreSQL database `camunda` in DB Cluster C
Replication	Inherited from Cluster C synchronous PostgreSQL replication
Worker access	HTTPS REST to `/engine-rest`
Auth	Basic auth or service auth over TLS, restricted to internal network
Endpoint	`https://camunda.sophix.internal/engine-rest`

Both Camunda nodes share the same database. If one node fails, the other continues from persisted workflow state.

6. Core Concepts

Concept	Meaning
BPMN file	XML process definition deployed to Camunda.
Process definition key	Stable workflow key used to start instances.
Process instance	One running workflow execution.
Business key	Domain aggregate ID, for example `orderId`, `subscriptionId`, or `workOrderId`.
Process variables	Small key/value data carried by the workflow. Store IDs and flags, not large payloads.
External task	Work item picked up by a module worker through `fetchAndLock`.
User task	Work item completed by a human through UI/module API.
Message catch	Workflow waits until a module correlates a named message.
Timer	Workflow waits until a duration/date. Handled by Camunda job executor.
Incident	Camunda-recorded failure needing retry, fix, or manual action.

7. BPMN File Layout

Each module owns its BPMN files under its codebase.

Recommended layout:

```
modules/{module-name}/
■■■ src/main/resources/
    ■■■ workflows/
        ■■■ {workflow-key}.bpmn
        ■■■ {workflow-key}-{operator}.bpmn
```

Examples:

```
modules/fulfillment-order-capture/src/main/resources/workflows/order-capture-wik.bpmn
modules/subscription-workflows/src/main/resources/workflows/sub-activate-wik.bpmn
modules/work-order/src/main/resources/workflows/workorder-installation-wik.bpmn
```

Rules:

Rule	Description
CAM-BPMN-1	One BPMN file per deployable workflow variant.
CAM-BPMN-2	Operator-specific behavior belongs in operator BPMN satellites or config rows.
CAM-BPMN-3	BPMN file names must be stable and lowercase kebab-case.
CAM-BPMN-4	`processDefinitionKey` must be stable; do not rename after production use.
CAM-BPMN-5	In-flight instances remain on the version they started with unless explicitly migrated.

8. Starting a Workflow

Start endpoint:

```
POST /engine-rest/process-definition/key/{processDefinitionKey}/start
Authorization: Basic <internal>
Content-Type: application/json

{
  "businessKey": "sub_01HX",
  "variables": {
    "subscriptionId": { "value": "sub_01HX", "type": "String" },
    "operatorCode": { "value": "WIK", "type": "String" },
    "correlationId": { "value": "corr_01HX", "type": "String" }
  }
}
```

Start rules:

Rule	Description
CAM-START-1	`businessKey` must be the domain aggregate ID.
CAM-START-2	Caller must persist its own domain/audit row before or in the same logical operation as workflow start.
CAM-START-3	Start requests must be idempotent at the owning module API layer.
CAM-START-4	Process variables must carry IDs and control flags only. Large payloads stay in module tables.

9. External Task Worker Pattern

Workers are module-owned long-running processes.

Fetch and lock:

```
POST /engine-rest/external-task/fetchAndLock
Authorization: Basic <internal>
Content-Type: application/json

{
  "workerId": "subscription-worker-1",
  "asyncResponseTimeout": 30000,
  "maxTasks": 10,
  "topics": [
    {
      "topicName": "sub-validate-preconditions",
      "lockDuration": 60000
    }
```

```
  ]
}
```

Complete:

```
POST /engine-rest/external-task/{taskId}/complete
Authorization: Basic <internal>
Content-Type: application/json

{
  "workerId": "subscription-worker-1",
  "variables": {
    "eligible": { "value": true, "type": "Boolean" }
  }
}
```

Failure:

```
POST /engine-rest/external-task/{taskId}/failure
Authorization: Basic <internal>
Content-Type: application/json

{
  "workerId": "subscription-worker-1",
  "errorMessage": "BIL-01 unavailable",
  "errorDetails": "HTTP 503 from billing",
  "retries": 3,
  "retryTimeout": 60000
}
```

Worker rules:

Rule	Description
CAM-WORKER-1	Workers must be idempotent. Camunda may retry a task.
CAM-WORKER-2	`workerId` must identify module and instance.
CAM-WORKER-3	Use one topic per business action, not one generic catch-all topic.
CAM-WORKER-4	Lock duration must be longer than expected task runtime.
CAM-WORKER-5	On transient failure, report failure with retries and retry timeout.
CAM-WORKER-6	On non-retryable business failure, complete with variables that route the BPMN to rejection/failure path, or throw BPMN error if modeled.

10. User Tasks

User tasks wait for a human action.

Rules:

Rule	Description
CAM-USER-1	Candidate groups must map to roles/groups in `FOUNDATION_AUTH.md` or EM/ICN-owned group catalogs.
CAM-USER-2	Users complete tasks through module APIs, not directly from arbitrary clients to Camunda.
CAM-USER-3	Module API must enforce authorization before forwarding task completion.
CAM-USER-4	User task payload must include enough context for UI display but not large documents.
CAM-USER-5	Every user task must have timeout/escalation behavior if business-critical.

Completion:

```
POST /engine-rest/task/{taskId}/complete
Authorization: Basic <internal>
Content-Type: application/json

{
  "variables": {
    "approvalDecision": { "value": "APPROVE", "type": "String" },
    "approvedBy": { "value": "user_01HX", "type": "String" }
  }
}
```

11. User Task Notification

Camunda creates the user task. ICN-01 or the owning module notifies the human.

Preferred pattern:

1. BPMN reaches user task.
2. Task create listener or module task scanner emits notification request.
3. ICN-01 sends internal notification to candidate users/groups.
4. User opens BO UI and completes task through module API.

Do not depend on email alone for critical user tasks. Use ICN-01 where available.

12. Message Correlation

Message catch events let a workflow wait for an external event.

Example:

```
POST /engine-rest/message
Authorization: Basic <internal>
Content-Type: application/json

{
  "messageName": "WorkOrderFinalized",
  "businessKey": "ord_01HX",
  "processVariables": {
    "workOrderId": { "value": "wo_01HX", "type": "String" },
    "finalOutcome": { "value": "COMPLETED", "type": "String" }
  }
}
```

Rules:

Rule	Description
CAM-MSG-1	Correlate by `businessKey` unless the owning DD explicitly defines another correlation key.
CAM-MSG-2	Message names use PascalCase business names.
CAM-MSG-3	Kafka consumers that correlate Camunda messages must be idempotent.
CAM-MSG-4	If no process instance matches, write to a DLQ/recovery table owned by the module and alert.
CAM-MSG-5	Do not lose unmatched messages; late-arriving workflow waits are normal in distributed systems.

13. Timers

Use timers for waits owned by the workflow:

- Payment timeout.
- KYC timeout.
- Work order timeout.
- Scheduled activation/resume.
- Escalation after no human action.

Rules:

Rule	Description
CAM-TIMER-1	Use ISO-8601 durations such as `PT7D`, `PT30M`, `PT48H`.
CAM-TIMER-2	Timer values that vary by operator must come from config variables.
CAM-TIMER-3	Boundary timers must route to explicit timeout handling.
CAM-TIMER-4	Long waits must be visible in module admin views.

14. Deployment and Versioning

Deployment endpoint:

```
POST /engine-rest/deployment/create
Content-Type: multipart/form-data
```

Deployment rules:

Rule	Description
CAM-DEP-1	CI must validate BPMN XML before deploy.
CAM-DEP-2	New workflow versions must not break existing in-flight instances.
CAM-DEP-3	Old BPMN versions are retained for in-flight workflows.
CAM-DEP-4	Production BPMN deployment must be traceable to Git commit/build ID.
CAM-DEP-5	Process definition keys are immutable once used in production.

15. Failure and Recovery

Failure	Behavior
One Camunda node down	Load balancer routes to the other node. Workflow state remains in DB.
Camunda DB unavailable	New starts, task operations, timers, and workers fail until DB recovers.
Worker dies while task locked	Lock expires; another worker can fetch the task.
Worker reports retries exhausted	Camunda incident is created. Ops must inspect and retry/fix.
Message arrives before workflow is waiting	Module must store unmatched message and retry correlation.
BPMN bug in deployed version	Stop new starts, deploy fixed version, decide whether to migrate or restart in-flight instances.

16. Monitoring

Alert on:

Signal	Threshold
Camunda node count	Less than 2
Camunda DB unavailable	Any
Incident count	More than 0 for critical workflows
External task backlog	Growing beyond normal baseline
External task retries exhausted	Any
Long-running instances	Exceed workflow-specific SLA
Failed message correlations	More than 0 sustained
Job executor failures	Any sustained failures

17. Medium Budget Infrastructure

17.1 Server Dimensions

Required medium baseline:

Server role	Count	CPU	RAM	Disk	Notes
Camunda app node	2	2-4 vCPU each	8 GB each	80-100 GB SSD each	Stateless app nodes; workflow state is in PostgreSQL.
Camunda PostgreSQL	Covered by `FOUNDATION_DB.md` Cluster C	Covered there	Covered there	Covered there	Do not deploy a separate DB outside Cluster C.
Load balancer	1 minimum, 2 preferred	1-2 vCPU	2 GB	20 GB SSD	NGINX/LB if on-prem.
Monitoring/log storage	Shared platform service	2 vCPU	4 GB	Depends on retention	Needed for incidents and worker logs.

Minimum acceptable start:

```
2 x Camunda nodes: 2 vCPU / 8 GB RAM / 80 GB SSD
1 x LB node:       1-2 vCPU / 2 GB RAM / 20 GB SSD
```

Preferred medium:

```
2 x Camunda nodes: 4 vCPU / 8-16 GB RAM / 100 GB SSD
2 x LB nodes:      1-2 vCPU / 2 GB RAM / 20 GB SSD
```

Placement rules:

- Do not place both Camunda nodes on the same physical host.
- Do not co-locate Camunda app nodes with Cluster C PostgreSQL primary if avoidable.
- Keep `/engine-rest` private. Only module services and admin networks should reach it.

17.2 Recommended OS

Use one operator-standard Linux distribution:

OS	Recommendation
Ubuntu Server 22.04 LTS or 24.04 LTS	Recommended default for medium teams.
Rocky Linux 9 / RHEL 9	Use if the operator standardizes on RHEL-family systems.

OS baseline:

- 64-bit minimal server install.
- No desktop packages.
- `chrony` or equivalent NTP enabled.
- Security patching through operator maintenance process.
- SSH restricted to admin VPN/network.
- Host firewall enabled.
- Java runtime pinned and tested with selected Camunda version.

17.3 Disk Partitions

For each Camunda app node with 80-100 GB disk:

```
/               30 GB
/opt/camunda    20 GB
/var/log        20 GB
/tmp            10 GB
free/reserved   remaining
```

Partition rules:

- Camunda app nodes should not store workflow state locally.
- Logs must not fill `/`.
- Workflow attachments, PDFs, and large payloads belong in module DB/object storage, not on Camunda node disk.

17.4 JVM Starting Point

For 8 GB RAM nodes:

```
JAVA_OPTS="-Xms2g -Xmx4g"
```

For 16 GB RAM nodes:

```
JAVA_OPTS="-Xms4g -Xmx8g"
```

Tune after observing external task backlog, job executor latency, and heap usage.

17.5 Network Ports

Flow	Port	Exposure
Module/API to Camunda LB	443	Internal application network only
LB to Camunda node	8080 or 8443	Internal only
Camunda to PostgreSQL/PgBouncer	6432	Internal only
Admin SSH	22	Admin VPN/network only

Flow	Port	Exposure
Monitoring scrape	Operator standard	Monitoring network only

18. AWS Deployment Recommendations

This section applies only when deploying Camunda on AWS.

18.1 Recommended AWS Shape

Component	AWS service	Recommendation
Camunda runtime	ECS on EC2/Fargate or EC2 Auto Scaling Group	ECS is preferred if the team already operates containers. EC2 ASG is simpler for VM-based teams.
Load balancer	Application Load Balancer	Internal ALB for `/engine-rest`; public access is not required.
Camunda DB	Amazon RDS for PostgreSQL	Use Cluster C RDS from `FOUNDATION_DB.md`, Multi-AZ.
Secrets	AWS Secrets Manager or SSM Parameter Store	Store DB credentials and Camunda internal auth secrets.
Logs	CloudWatch Logs	Ship Camunda app logs and worker logs.
Metrics	CloudWatch/Prometheus	Track incidents, task backlog, JVM, DB connection pool.

18.2 Compute Sizing

ECS/Fargate medium start:

```
Camunda service desired count: 2 tasks
Task size:                     2 vCPU / 8 GB memory
Placement:                     spread across 2 Availability Zones
```

EC2 ASG medium start:

```
2 x EC2 instances: m7g.large or m6i.large
EBS:               80-100 GB gp3 each
```

Preferred medium:

```
2 x ECS tasks or EC2 nodes: 4 vCPU / 8-16 GB memory
```

Use Graviton only after verifying the selected Camunda/JDK/container image runs cleanly on ARM. Otherwise use x86 instances.

18.3 AWS Network Layout

Recommended VPC layout:

```
Private app subnets:
  - Camunda tasks/instances
  - SOPHIX module services

Private DB subnets:
  - RDS PostgreSQL Cluster C

Public subnets:
  - No Camunda resources unless an admin UI is intentionally exposed through VPN-protected access
```

Security group rules:

Source	Destination	Port	Purpose
Module service SG	Internal ALB SG	443	Camunda REST
ALB SG	Camunda task/EC2 SG	8080 or 8443	Forward traffic
Camunda SG	RDS/PgBouncer SG	5432 or 6432	DB access
Admin VPN/bastion/SSM	Camunda nodes	22 or SSM	Admin access if EC2

18.4 ALB and Health Checks

Use an internal Application Load Balancer:

- Health check path: a Camunda/app health endpoint.
- Health check must fail if the node cannot reach the Camunda DB.
- Keep `/engine-rest` reachable only from internal service networks.
- Enable access logs if troubleshooting workflow traffic.

18.5 AWS Operational Notes

- Store BPMN artifacts in the application build artifact or container image.
- Deploy BPMN through CI/CD, not manual console uploads.
- Take an RDS snapshot before major Camunda upgrades.
- Use CloudWatch alarms for unhealthy targets, task restarts, RDS latency, and application errors.
- If using ECS, configure deployment minimum healthy percent so one task remains live during rolling deploys.

Reference docs:

- [Amazon ECS with Application Load Balancer](https://docs.aws.amazon.com/AmazonECS/latest/developerguide/alb.html)
- [Application Load Balancer target health checks](https://docs.aws.amazon.com/elasticloadbalancing/latest/application/target-group-health-checks.html)
- [Amazon RDS Multi-AZ deployments](https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.html)

19. Developer Checklist

- [ ] Workflow has a stable process definition key.
- [ ] `businessKey` is the owning aggregate ID.
- [ ] Process variables contain IDs/flags, not large payloads.
- [ ] Every external task has an owning module worker.
- [ ] Workers are idempotent.
- [ ] Failure/retry behavior is modeled.
- [ ] User tasks have candidate groups and authorization checks.
- [ ] Message catch events have deterministic correlation.
- [ ] Timers have explicit timeout handling.
- [ ] BPMN XML is validated in CI.
- [ ] Deployment is traceable to Git/build ID.
- [ ] Admin/ops view exists for stuck or long-running workflow instances.